

Projet d'Administration des Systèmes et des Réseaux II

Architecture de réseaux avec Cumulus et Debian

Tony LENG, 20221861 & Paul VEROT, 20212888

Master I CNS-SR, 2026

Semestre 2



UNIVERSITÉ ÉVRY
PARIS-SACLAY



UNIVERSITÉ ÉVRY

PARIS-SACLAY

Département d'informatique
univ-evry.fr

Titre:

Projet d'Administration des Systèmes et des Réseaux II

Thème:

Architecture de réseaux avec Cumulus et Debian

Groupe:

Master I CNS-SR

Participants:

Tony LENG, 20221861

Paul VEROT, 20212888

Email:

lengtony91@gmail.com

20221861@etud.univ-evry.fr

paul@paulverot.fr

20212888@etud.univ-evry.fr

Superviseur:

Mehdi Denou

Nombre de pages:

34

Dernier changement:

03-04-2026

Résumé:

Ce projet porte sur la mise en place de quatre architectures et sur l'étude des protocoles associés. Nous utiliserons Cumulus Linux et Debian 13 pour les switch et client respectivement. Notamment, le projet porte sur la redondance des liens et des switch ainsi que sur Spanning Tree.

Chapitres

1 Partage Réseau NFS	3
1.1 Résumé des contraintes et des demandes:	3
1.2 Mise en Place	4
1.2.1 Sur VMware	4
1.2.2 Sur les machines :	4
1.2.3 Explication des Commandes	7
1.3 Configuration des Adresses IP sur les Machines	8
1.3.1 Clients	8
1.3.2 Serveur NFS	9
1.3.2.1 Explication des paramètres du Bond	9
1.4 Configuration du Pare-feu	10
1.4.1 Sur Switch-3	10
1.4.1.1 Explication des règles de pare-feu	10
1.5 Configuration du Partage NFS	11
1.5.1 Sur le Serveur NFS	11
1.5.1.1 Test du transfert NFS	11
1.5.1.2 Test d'accès restreint	12
2 Redondance de liens	13
2.1 Résumé des contraintes et des demandes	13
2.2 Problème de la topologie initiale	14
2.3 Mise en place	15
2.3.1 Sur VMWare	15
2.3.2 Sur les Machines	15
2.3.2.1 Rôle et état des ports	15
2.4 Test STP	16
2.5 Configuration de RSTP	18
2.6 Test RSTP	18
3 Tolérance de panne, couche "access"	20
3.1 Résumé des contraintes et des demandes	20
3.2 Mise en place	21
3.2.1 Sur VMware	21
3.2.2 Sur les machines	21
3.3 Test	25
4 Redondance de Routeurs	26
4.1 Architecture	26
4.2 Mise en Place	27
4.2.1 Sur VMware	27
4.2.2 Sur les Machines	27

4.3 Tests	31
4.3.1 Coupure dans la couche Access	31
4.3.2 Coupure dans la couche Disribution	32

PARTAGE RÉSEAU NFS

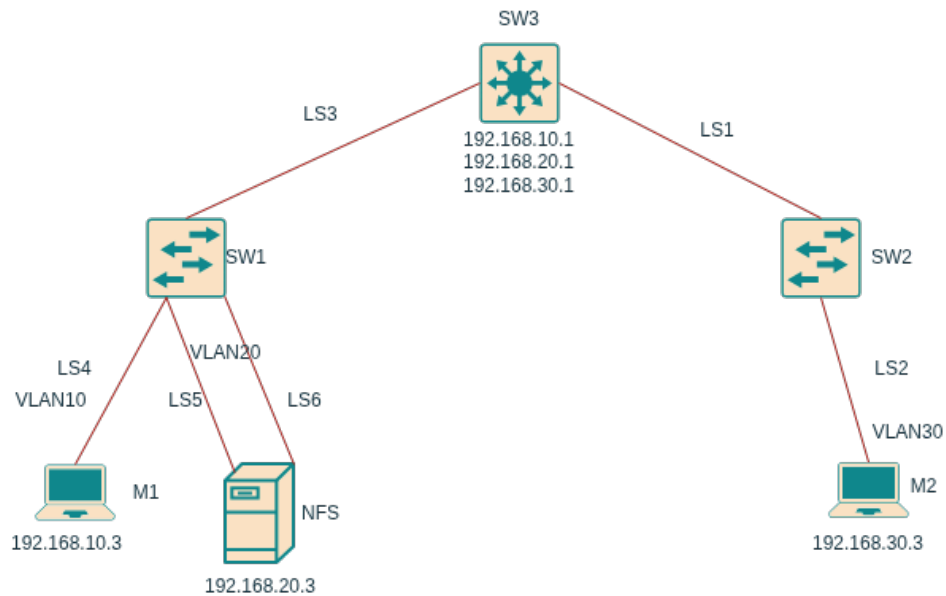


Figure 1.1: Maquette du réseau.

1.1 RÉSUMÉ DES CONTRAINTES ET DES DEMANDES:

- Les Switchs 1 et 2 n'utilisent que des fonctions de niveau 2.
- Le Switch 3 peut utiliser des fonctions de niveau 3.
- Les machines 1 et 2 doivent être isolées.
- Les machines 1 et 2 doivent avoir accès au serveur NFS.
- Les machines 1 et 2 doivent, pour transmettre via NFS au serveur, uniquement dans leurs dossiers respectifs.
- Tolérance à la panne sur les interfaces du serveur NFS.

Plan :

- Pour assurer l'isolation des machines, elles seront placées dans des VLANs différents et la communication entre ces réseaux sera interdite par des règles de pare-feu sur le Switch-3, qui agira comme RPD pour les deux machines. On peut également placer des règles pour restreindre les types de communications entre les machines et le serveur NFS (couper SSH, http, https, ...).

- Pour garantir la disponibilité des liens sur le serveur NFS, deux interfaces seront configurées en LACP mode **actif-backup** et connectées au Switch-1. En cas de défaillance du lien actif, le second lien prendra le relais.
- Pour le serveur NFS, `nfs-kernel-server` devra être configuré pour créer des partages réseau et définir les modalités de montage pour les clients.

1.2 MISE EN PLACE

On a besoin de mettre en place 3 VLANs, un pare-feu, une agrégation de liens et un serveur NFS.

1.2.1 SUR VMWARE

M1	SW1
adapter 1 : host only	adapter 1 : host only
adapter 2 : lan segment 4	adapter 2 : lan segment 3
M2	adapter 3 : lan segment 4
adapter 1 : host only	adapter 4 : lan segment 5
adapter 2 : lan segment 2	adapter 5 : lan segment 6
NFS	SW2
adapter 1 : host only	adapter 1 : host only
adapter 2 : lan segment 5	adapter 2 : lan segment 1
adapter 3 : lan segment 6	adapter 3 : lan segment 2
	SW3
	adapter 1 : host only
	adapter 2 : lan segment 1
	adapter 3 : lan segment 3

Toutes les machines hors switch sont des VM Debian 13 n'ayant pas de Display Manager installé. Chacune avec 2 coeurs et 677 MiB de RAM.

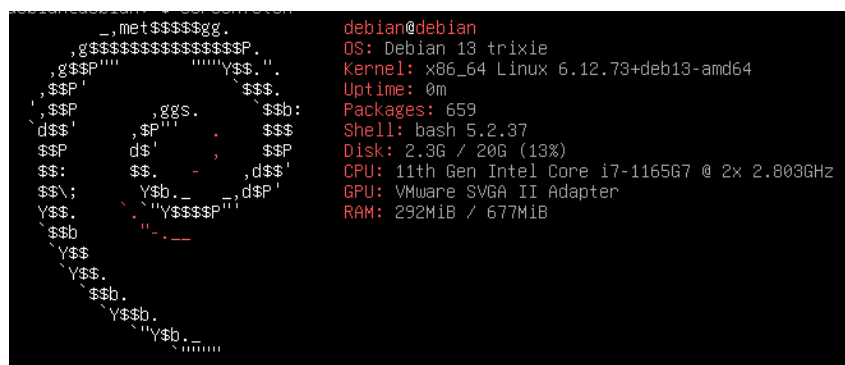


Figure 1.2: Affichage des spécifications avec screenfetch

1.2.2 SUR LES MACHINES :

Configuration des VLAN :

Switch-1

```

net add hostname sw1
net add bridge bridge ports swp1,swp2,swp3,swp4
net add bridge bridge vlan-aware
net add interface swp2 bridge access 10
net add interface swp3,swp4 bridge access 20
net add interface swp1 bridge trunk vlans 10,20
net commit

```

```

cumulus@sw1:mgmt:~$ net show interface
State  Name   Spd  MTU   Mode      LLDP      Summary
-----
UP     lo      N/A  65536  Loopback
lo
UP     eth0    10G  1500  Mgmt      sw2 (eth0) Master: mgmt(UP)
eth0      IP: 192.168.85.142/24(DHCP)
UP     swp1    1G   9216  Trunk/L2  sw3 (swp2) Master: bridge(UP)
UP     swp2    1G   9216  Access/L2 Master: bridge(UP)
UP     swp3    1G   9216  Access/L2 Master: bridge(UP)
UP     swp4    1G   9216  Access/L2 Master: bridge(UP)
UP     bridge  N/A   9216  Bridge/L2
UP     mgmt    N/A  65536  VRF
mgmt      IP: 127.0.0.1/8
          IP: ::1/128

```

```

cumulus@sw1:mgmt:~$ net show bridge vlan

```

Interface	VLAN	Flags
swp1	1	PVID, Egress Untagged
	10	
	20	
swp2	10	PVID, Egress Untagged
swp3	20	PVID, Egress Untagged
swp4	20	PVID, Egress Untagged

Switch-2

```

net add hostname sw2
net add bridge bridge ports swp1,swp2
net add bridge bridge vlan-aware
net add interface swp2 bridge access 30
net add interface swp1 bridge trunk vlans 20,30
net commit

```

```

cumulus@sw2:mgmt:~$ net show interface
State  Name   Spd  MTU   Mode      LLDP      Summary
-----
UP     lo      N/A  65536  Loopback
lo
IP: 127.0.0.1/8
IP: ::1/128

```

```

UP    eth0    10G  1500  Mgmt      sw1 (eth0) Master: mgmt(UP)
      eth0                      IP: 192.168.85.141/24(DHCP)
UP    swp1    1G   9216  Trunk/L2  sw3 (swp1) Master: bridge(UP)
UP    swp2    1G   9216  Access/L2                Master: bridge(UP)
UP    bridge N/A   9216  Bridge/L2
UP    mgmt    N/A  65536  VRF                      IP: 127.0.0.1/8
      mgmt                      IP: ::1/128

```

```
cumulus@sw2:mgmt:~$ net show bridge vlan
```

```

Interface  VLAN  Flags
-----
swp1       1    PVID, Egress Untagged
           20
           30
swp2       30    PVID, Egress Untagged

```

Switch-3

```

net add hostname sw3

net add bridge bridge ports swp1,swp2
net add bridge bridge vlan-aware

net add interface swp2 bridge trunk vlans 10,20
net add interface swp1 bridge trunk vlans 20,30

net add vlan 10 ip address 192.168.10.1/24
net add vlan 20 ip address 192.168.20.1/24
net add vlan 30 ip address 192.168.30.1/24
net commit

```

```

cumulus@sw3:mgmt:~$ net show interface
State  Name    Spd  MTU   Mode      LLDP      Summary
-----
UP     lo      N/A  65536  Loopback
      lo
UP     eth0    10G  1500  Mgmt      sw1 (eth0) Master: mgmt(UP)
      eth0                      IP: 192.168.85.140/24(DHCP)
UP     swp1    1G   9216  Trunk/L2  sw2 (swp1) Master: bridge(UP)
UP     swp2    1G   9216  Trunk/L2  sw1 (swp1) Master: bridge(UP)
UP     bridge N/A   9216  Bridge/L2
UP     mgmt    N/A  65536  VRF                      IP: 127.0.0.1/8
      mgmt                      IP: ::1/128
UP     vlan10  N/A  9216  Interface/L3             IP: 192.168.10.1/24
UP     vlan20  N/A  9216  Interface/L3             IP: 192.168.20.1/24
UP     vlan30  N/A  9216  Interface/L3             IP: 192.168.30.1/24

```

```
cumulus@sw3:mgmt:~$ net show bridge vlan
```

Interface	VLAN	Flags
swp1	1	PVID, Egress Untagged
	20	
	30	
swp2	1	PVID, Egress Untagged
	10	
	20	
bridge	10	
	20	
	30	

1.2.3 EXPLICATION DES COMMANDES

- `net add hostname sw1`
 - Définit le nom d'hôte du switch.
- `net add bridge bridge ports swp1,swp2,swp3,swp4`
 - Ajoute les interfaces physiques (swp1 à swp4) au bridge principal.
- `net add bridge bridge vlan-aware`
 - Active le mode "VLAN-aware" du bridge pour permettre la gestion des VLANs.
- `net add interface swp2 bridge access 10`
`net add interface swp3,swp4 bridge access 20`
 - Configure les interfaces en mode accès:
 - swp2 appartient au VLAN 10.
 - swp3 et swp4 appartiennent au VLAN 20.
- `net add interface swp1 bridge trunk vlans 10,20`
 - Configure swp1 en mode trunk pour transporter les VLANs 10 et 20.
- `net add vlan 10 ip address 192.168.10.1/24`
`net add vlan 20 ip address 192.168.20.1/24`
`net add vlan 30 ip address 192.168.30.1/24`
 - Crée les interfaces VLAN et assigne les adresses IP de gestion:
 - VLAN 10: 192.168.10.0/24
 - VLAN 20: 192.168.20.0/24
 - VLAN 30: 192.168.30.0/24
- `net commit`
 - Valide et applique toutes les modifications précédentes, analogue à un commit sur Git.

1.3 CONFIGURATION DES ADRESSES IP SUR LES MACHINES

1.3.1 CLIENTS

Machine 1

Dans le fichier

/etc/network/interfaces :

```
auto ens36
iface ens36 inet static
    address 192.168.10.3/24
    gateway 192.168.10.1
```

```
root@debian:~# ip a
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host noprefixroute
        valid_lft forever preferred_lft forever
2: ens33: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 00:0c:29:3b:fa:96 brd ff:ff:ff:ff:ff:ff
    altname enp2s1
    altname enx000c293bfa96
    inet 192.168.59.146/24 brd 192.168.59.255 scope global dynamic noprefixroute ens33
        valid_lft 1683sec preferred_lft 1458sec
    inet6 fe80::87ea:ca07:2889:30ea/64 scope link
        valid_lft forever preferred_lft forever
3: ens36: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 00:0c:29:3b:fa:a0 brd ff:ff:ff:ff:ff:ff
    altname enp2s4
    altname enx000c293bfaa0
    inet 192.168.10.3/24 brd 192.168.10.255 scope global ens36
        valid_lft forever preferred_lft forever
    inet6 fe80::20c:29ff:fe3b:faa0/64 scope link proto kernel_ll
        valid_lft forever preferred_lft forever
root@debian:~#
```

Figure 1.3: Configuration IP de la Machine 1.

Machine 2

```
auto ens36
iface ens36 inet static
    address 192.168.30.3/24
    gateway 192.168.30.1
```

```
root@debian:~# ip a
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host noprefixroute
        valid_lft forever preferred_lft forever
2: ens33: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 00:0c:29:db:e3:a3 brd ff:ff:ff:ff:ff:ff
    altname enp2s1
    altname enx000c29dbe3a3
    inet 192.168.59.145/24 brd 192.168.59.255 scope global dynamic noprefixroute ens33
        valid_lft 1687sec preferred_lft 1462sec
    inet6 fe80::6a89:ec8e:78c2:74c8/64 scope link
        valid_lft forever preferred_lft forever
3: ens36: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 00:0c:29:db:e3:ad brd ff:ff:ff:ff:ff:ff
    altname enp2s4
    altname enx000c29dbe3ad
    inet 192.168.30.3/24 brd 192.168.30.255 scope global ens36
        valid_lft forever preferred_lft forever
    inet6 fe80::20c:29ff:fedb:e3ad/64 scope link proto kernel_ll
        valid_lft forever preferred_lft forever
root@debian:~# _
```

Figure 1.4: Configuration IP de la Machine-2

1.3.2 SERVEUR NFS

```
auto bond0
iface bond0 inet static
    address 192.168.20.3/24
    gateway 192.168.20.1
    bond-slaves ens36 ens37
    bond-mode active-backup
    bond-miimon 100
    bond-primary eth1

auto ens36
iface ens36 inet manual
    bond-master bond0

auto ens37
iface ens37 inet manual
    bond-master bond0
```

1.3.2.1 EXPLICATION DES PARAMÈTRES DU BOND

- Définir une adresse.
- Définir une passerelle par défaut (les trois machines ont comme passerelle par défaut le Switch-3).
- Indiquer les interfaces membres de l'agrégation.
- Indiquer le mode d'agrégation.
- Indiquer le délai de bascule.
- Indiquer l'interface principale (celle qui serait UP par défaut).
- Indiquer le maître d'agrégation aux interfaces.

```
root@debian:~# ip a
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host noprefixroute
        valid_lft forever preferred_lft forever
2: ens33: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 00:0c:29:ab:a8:67 brd ff:ff:ff:ff:ff:ff
    altname enp2s1
    altname enx00c29aba867
    inet 192.168.59.147/24 brd 192.168.59.255 scope global dynamic noprefixroute ens33
        valid_lft 1720sec preferred_lft 1495sec
    inet6 fe80::4ab0:1534:cd8c:c5a6/64 scope link
        valid_lft forever preferred_lft forever
3: ens36: <BROADCAST,MULTICAST,SLAVE,UP,LOWER_UP> mtu 1500 qdisc fq_codel master bond0 state UP group default qlen 1000
    link/ether 00:0c:29:ab:a8:71 brd ff:ff:ff:ff:ff:ff
    altname enp2s4
    altname enx00c29aba871
    inet6 fe80::c21c:7a59:d86e:84f3/64 scope link tentative noprefixroute
        valid_lft forever preferred_lft forever
4: ens37: <BROADCAST,MULTICAST,SLAVE,UP,LOWER_UP> mtu 1500 qdisc fq_codel master bond0 state UP group default qlen 1000
    link/ether 00:0c:29:ab:a8:71 brd ff:ff:ff:ff:ff:ff permaddr 00:0c:29:ab:a8:7b
    altname enp2s5
    altname enx00c29aba87b
    inet6 fe80::6151:7189:7316:a397/64 scope link tentative noprefixroute
        valid_lft forever preferred_lft forever
5: bond0: <BROADCAST,MULTICAST,MASTER,UP,LOWER_UP> mtu 1500 qdisc noqueue state UP group default qlen 1000
    link/ether 00:0c:29:ab:a8:71 brd ff:ff:ff:ff:ff:ff
    inet 192.168.20.3/24 brd 192.168.20.255 scope global bond0
        valid_lft forever preferred_lft forever
    inet6 fe80::20c:29ff:feab:a871/64 scope link proto kernel_l1
        valid_lft forever preferred_lft forever
root@debian:~#
```

Figure 1.5: Configuration IP du Serveur-NFS

On peut vérifier l'état du Bond : `cat /proc/net/bonding/bond0`

```

root@debian:~# cat /proc/net/bonding/bond0
Ethernet Channel Bonding Driver: v6.12.73+deb13-amd64

Bonding Mode: fault-tolerance (active-backup)
Primary Slave: None
Currently Active Slave: ens36
MII Status: up
MII Polling Interval (ms): 100
Up Delay (ms): 0
Down Delay (ms): 0
Peer Notification Delay (ms): 0

Slave Interface: ens36
MII Status: up
Speed: 1000 Mbps
Duplex: full
Link Failure Count: 0
Permanent HW addr: 00:0c:29:ab:a8:71
Slave queue ID: 0

Slave Interface: ens37
MII Status: up
Speed: 1000 Mbps
Duplex: full
Link Failure Count: 0
Permanent HW addr: 00:0c:29:ab:a8:7b
Slave queue ID: 0
root@debian:~# █

```

Figure 1.6: Vérification de l'état de l'agrégation

1.4 CONFIGURATION DU PARE-FEU

1.4.1 SUR SWITCH-3

Dans `/etc/cumulus/acl/policy.d/50_custom.rules`

```

[iptables]
-A FORWARD -s 192.168.30.3 -d 192.168.10.3 -j DROP
-A FORWARD -s 192.168.10.3 -d 192.168.30.3 -j DROP

```

1.4.1.1 EXPLICATION DES RÈGLES DE PARE-FEU

Ces règles ont pour but de détruire tout les paquets entre les réseaux .10.0/24 et .30.0/24 ; ainsi, les deux machines clients ne pourront pas communiquer entre elles.

On peut tester que les machines ne peuvent pas se pinguer :

```

3: ens36: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 00:0c:29:db:e3:ad brd ff:ff:ff:ff:ff:ff
    altname enp2s4
    altname enx000c29dbe3ad
    inet 192.168.30.3/24 brd 192.168.30.255 scope global ens36
        valid_lft forever preferred_lft forever
    inet6 fe80::20c:29ff:fedb:e3ad/64 scope link proto kernel_l1
        valid_lft forever preferred_lft forever
root@debian:~# ping 192.168.20.3
PING 192.168.20.3 (192.168.20.3) 56(84) bytes of data.
64 bytes from 192.168.20.3: icmp_seq=1 ttl=63 time=1.08 ms
64 bytes from 192.168.20.3: icmp_seq=2 ttl=63 time=2.14 ms
64 bytes from 192.168.20.3: icmp_seq=3 ttl=63 time=2.53 ms
64 bytes from 192.168.20.3: icmp_seq=4 ttl=63 time=1.03 ms
^C
--- 192.168.20.3 ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3005ms
rtt min/avg/max/mdev = 1.026/1.695/2.529/0.654 ms
root@debian:~# ping 192.168.10.3
PING 192.168.10.3 (192.168.10.3) 56(84) bytes of data.
^C
--- 192.168.10.3 ping statistics ---
19 packets transmitted, 0 received, 100% packet loss, time 18427ms

root@debian:~# _

```

Figure 1.7: Test de communication entre les machines.

On peut voir que la Machine-2 peut communiquer avec le serveur NFS (.20.0/24) mais pas avec la Machine-1

1.5 CONFIGURATION DU PARTAGE NFS

1.5.1 SUR LE SERVEUR NFS

```
apt install nfs-kernel-server

mkdir /machine1
mkdir /machine2

chown nobody:nogroup /machine1 /machine2
chmod 777 /machine1 /machine2
```

On ajoute dans /etc/exports :

```
/machine1 192.168.10.0/24(rw,sync,no_subtree_check)
/machine2 192.168.30.0/24(rw,sync,no_subtree_check)
```

Ensuite :

```
exportfs -a
systemctl restart nfs-kernel-server
```

```
# Machine-1
mkdir /mnt/backup_m1
mount -t nfs -o vers=4 192.168.20.3:/machine1 /mnt/backup_m1
```

1.5.1.1 TEST DU TRANSFERT NFS

On peut ensuite tester que le transfert fonctionne en créant un fichier dans /mnt/backup_m1 et vérifier qu'il apparaît bien sur le serveur NFS.

```
# Machine-1
touch /mnt/backup_m1/test.ms

# Serveur NFS
ls -la /machine1
```

```
root@debian:~# mount -t nfs -o vers=4 192.168.20.3:/machine1 /mnt/backup_m1
root@debian:~# ls /mnt/backup_m1/
root@debian:~# lsblk
NAME        MAJ:MIN RM  SIZE RO TYPE MOUNTPOINTS
sda         8:0      0   20G  0 disk /
|-sda1      8:1      0  19.2G  0 part /
|--sda2     8:2      0    1K  0 part /
|--sda5     8:5      0   766M  0 part [SWAP]
sdb         11:0     1    754M  0 rom
root@debian:~# touch test.ms /mnt/backup_m1/
root@debian:~# touch /mnt/backup_m1/test.ms
root@debian:~# _

~
~
~
/etc/exports
root@debian:~# ls -la /machine1
total 8
drwxr-xr-x  2 nobody nogroup 4096 Mar  9 21:32 .
drwxr-xr-x 20 root    root    4096 Mar  9 21:26 ..
-rw-rw-r--  1 nobody nogroup   0 Mar  9 21:32 test.ms
root@debian:~#
```

Figure 1.8: En haut, la création du fichier test.ms dans le dossier /mnt/backup_m1 sur la Machine-1. En bas, on constate la présence de test.ms dans /machine1 sur le Serveur-NFS.

Pour vérifier que le transfert continuera même si un des liens tombait, j'ai créé un fichier de 1Gb avec la commande `fallocate -l 1G big.txt`, puis je l'ai copié dans le dossier `/mnt/backup_m1/`, après quelques instants, j'ai coupé l'interface `swp3` sur le Switch-1. Ainsi, le transfert devrait passer sur l'autre lien.

```

debian@debian:~$ sh
sh                sha224sum      sha384sum        sharkd           shift            showconsolefont  shred
sha1sum          sha256sum      sha512sum        shasum          shopt            showkey          shuf
debian@debian:~$ sha256sum /machine1/big.txt
49bc20df15e412a64472421e13fe86ff1c5165e18b2afccf160d4dc19fe68a14  /machine1/big.txt
debian@debian:~$
sha224sum sha256sum
debian@debian:~$ sha256sum big.txt
49bc20df15e412a64472421e13fe86ff1c5165e18b2afccf160d4dc19fe68a14  big.txt
debian@debian:~$

```

Figure 1.9: sha256 des fichiers locaux (en bas) et distant (en haut)

On peut constater sur le serveur que le fichier est bien apparu mais également que le checksum est identique, donc tout les bits ont correctement été transmis.

1.5.1.2 TEST D'ACCÈS RESTREINT

On peut aussi vérifier que l'on ne peut pas monter le partage (share) si l'on ne fait pas partie du réseau désigné :

```

# Machine-2
mkdir /mnt/backup_m2
mount -t nfs -o vers=4 192.168.20.3:/machine1 /mnt/backup_m2

```

```

^C
--- 192.168.10.3 ping statistics ---
3 packets transmitted, 3 received, 0% packet loss, time 2004ms
rtt min/avg/max/mdev = 1.416/2.210/2.764/0.576 ms
debian@debian:~$ su -
Password:
root@debian:~# mkdir /mnt/backup_m2
root@debian:~# mount -t nfs -o vers=4 192.168.20.3:/machine1 /mnt/backup_m2/
mount.nfs: mounting 192.168.20.3:/machine1 failed, reason given by server: No such file or directory
root@debian:~# mount -t nfs -o vers=4 192.168.20.3:/machine2 /mnt/backup_m2/
root@debian:~#

```

Figure 1.10: Échec de la tentative de montage d'un partage non autorisé.

Comme attendu, la Machine-2 ne peut se connecter qu'à son partage et pas à celui dédié à la Machine-1.

REDONDANCE DE LIENS

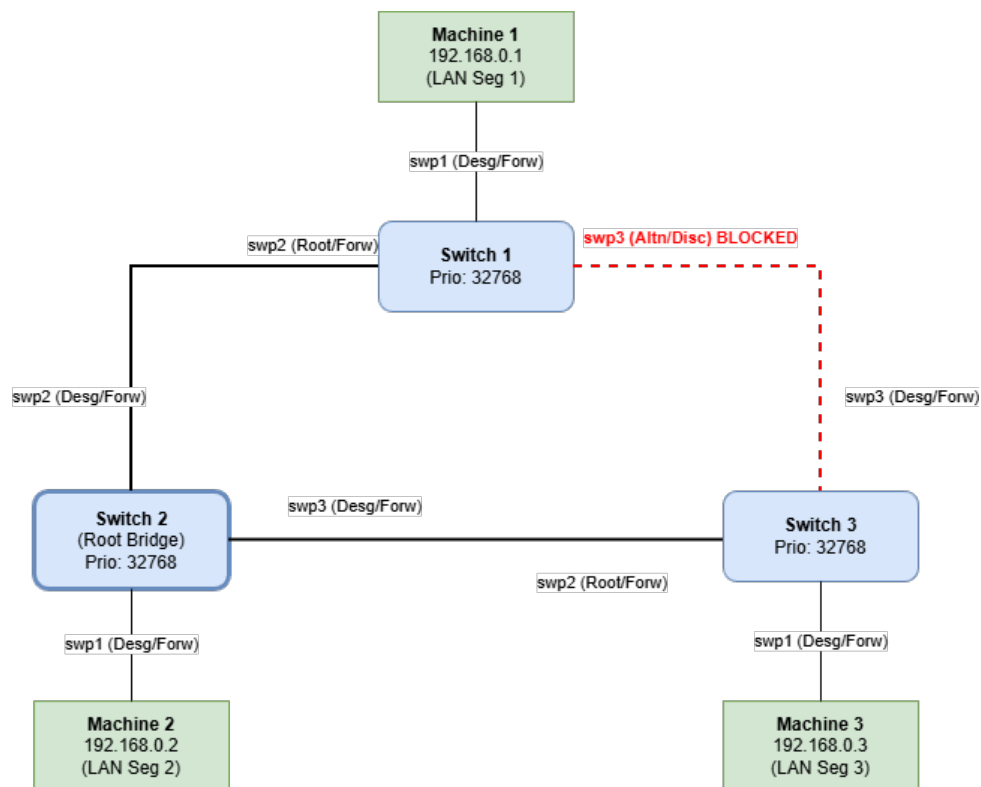


Figure 2.1: Schéma du Réseau

2.1 RÉSUMÉ DES CONTRAINTES ET DES DEMANDES

- Les trois switch (Switch 1, 2 et 3) n'utilisent que des fonctions de niveau 2.
- Il est impératif de mettre en place un réseau redondant entre les trois switch tout en garantissant l'absence de boucle L2.
- La solution doit permettre une reprise du trafic la plus rapide possible en cas de coupure d'un lien.

Plan :

- Pour éviter les tempêtes de diffusion (broadcast storms) inhérentes à notre topologie physique en anneau, nous allons forcer l'utilisation du protocole STP (Spanning Tree Protocol) standard sur les trois commutateurs. Il bloquera logiquement l'un des chemins redondants pour briser la boucle L2 tout en le gardant en secours.
- Pour évaluer la résilience de cette première configuration, nous simulerons une panne en coupant le lien actif. L'objectif sera d'observer le délai de bascule d'environ 30 secondes, causé par les

états intermédiaires temporisés (Listening et Learning) imposés par les timers par défaut du STP classique.

- Pour répondre à la contrainte d'une reprise du trafic la plus rapide possible, nous ferons ensuite évoluer la configuration pour utiliser le protocole RSTP (Rapid Spanning Tree Protocol, 802.1w). Grâce à son mécanisme de négociation active entre les switches, le port de secours passera immédiatement à l'état Forwarding en cas de nouvelle coupure, assurant ainsi une continuité de service quasi parfaite.

2.2 PROBLÈME DE LA TOPOLOGIE INITIALE

Afin d'illustrer la criticité d'une topologie physique en anneau non gérée, nous avons désactivé la prévention de boucles sur nos switch (en ajoutant la ligne `bridge-stp off` dans `/etc/network/interfaces`).

```
auto bridge
iface bridge
    bridge-ports swp1 swp2 swp3
    bridge-vlan-aware yes
    bridge-stp off
```

Nous avons ensuite vidé le cache ARP de la Machine 1 et lancé un test de ping vers la Machine 2, la forçant à en envoyer un nouveau.

```
debian@debian:~$ ping 192.168.0.2
PING 192.168.0.2 (192.168.0.2) 56(84) bytes of data.
64 bytes from 192.168.0.2: icmp_seq=93 ttl=64 time=3.61 ms
64 bytes from 192.168.0.2: icmp_seq=118 ttl=64 time=2.18 ms
```

```
17:16:28.876363 ARP, Request who-has 192.168.0.2 tell 192.168.0.1, length 46
17:16:28.876372 ARP, Reply 192.168.0.2 is-at 00:0c:29:d3:6a:7d (oui Unknown), length 28
17:16:28.876757 ARP, Request who-has 192.168.0.2 tell 192.168.0.1, length 46
17:16:28.876767 ARP, Reply 192.168.0.2 is-at 00:0c:29:d3:6a:7d (oui Unknown), length 28
17:16:28.877607 ARP, Request who-has 192.168.0.2 tell 192.168.0.1, length 46
17:16:28.877609 ARP, Request who-has 192.168.0.2 tell 192.168.0.1, length 46
17:16:28.877618 ARP, Reply 192.168.0.2 is-at 00:0c:29:d3:6a:7d (oui Unknown), length 28
17:16:28.877686 ARP, Reply 192.168.0.2 is-at 00:0c:29:d3:6a:7d (oui Unknown), length 28
17:16:28.878612 ARP, Request who-has 192.168.0.2 tell 192.168.0.1, length 46
17:16:28.878622 ARP, Reply 192.168.0.2 is-at 00:0c:29:d3:6a:7d (oui Unknown), length 28
17:16:28.879511 ARP, Request who-has 192.168.0.2 tell 192.168.0.1, length 46
17:16:28.879512 ARP, Request who-has 192.168.0.2 tell 192.168.0.1, length 46
17:16:28.879524 ARP, Reply 192.168.0.2 is-at 00:0c:29:d3:6a:7d (oui Unknown), length 28
17:16:28.879616 ARP, Reply 192.168.0.2 is-at 00:0c:29:d3:6a:7d (oui Unknown), length 28
17:16:28.881026 ARP, Request who-has 192.168.0.2 tell 192.168.0.1, length 46
17:16:28.881037 ARP, Reply 192.168.0.2 is-at 00:0c:29:d3:6a:7d (oui Unknown), length 28
17:16:28.881896 ARP, Request who-has 192.168.0.2 tell 192.168.0.1, length 46
17:16:28.881906 ARP, Reply 192.168.0.2 is-at 00:0c:29:d3:6a:7d (oui Unknown), length 28
17:16:28.882750 ARP, Request who-has 192.168.0.2 tell 192.168.0.1, length 46
17:16:28.882758 ARP, Reply 192.168.0.2 is-at 00:0c:29:d3:6a:7d (oui Unknown), length 28
17:16:28.883133 ARP, Request who-has 192.168.0.2 tell 192.168.0.1, length 46
17:16:28.883143 ARP, Reply 192.168.0.2 is-at 00:0c:29:d3:6a:7d (oui Unknown), length 28
17:16:28.884598 ARP, Request who-has 192.168.0.2 tell 192.168.0.1, length 46
17:16:28.884609 ARP, Reply 192.168.0.2 is-at 00:0c:29:d3:6a:7d (oui Unknown), length 28
17:16:28.884989 ARP, Request who-has 192.168.0.2 tell 192.168.0.1, length 46
17:16:28.884998 ARP, Reply 192.168.0.2 is-at 00:0c:29:d3:6a:7d (oui Unknown), length 28
17:16:28.885404 ARP, Request who-has 192.168.0.2 tell 192.168.0.1, length 46
17:16:28.885406 ARP, Request who-has 192.168.0.2 tell 192.168.0.1, length 46
17:16:28.885415 ARP, Reply 192.168.0.2 is-at 00:0c:29:d3:6a:7d (oui Unknown), length 28
17:16:28.885478 ARP, Reply 192.168.0.2 is-at 00:0c:29:d3:6a:7d (oui Unknown), length 28
^C
21191 packets captured
46651 packets received by filter
25284 packets dropped by kernel
debian@debian:~$
```

Figure 2.2: tcpdump ne montrant que les trames ARP

Explication du problème : L'absence d'un protocole d'évitement de boucle provoque presque instantanément un effondrement du réseau.

Broadcast Storm : La requête ARP (Broadcast) initiale générée par le ping s'est engouffrée dans les liens physiques redondants. Comme le montre le tcpdump, cette trame a été dupliquée et amplifiée à

l'infini. En une fraction de seconde, le système a généré des dizaines de milliers de paquets identiques.

Saturation des ressources : Cette tempête sature instantanément les tampons mémoire (buffers) des équipements. Plus de 25 000 paquets ont dû être détruits par le noyau Linux (dropped by kernel) pour éviter un crash total du système.

Impact sur le trafic : Le réseau est devenu inutilisable. Le trafic légitime est noyé par la tempête, entraînant des sauts massifs dans les séquences ICMP (de seq=93 à seq=118).

Ce qui montre qu'un protocole de niveau 2 (STP/RSTP) est indispensable pour bloquer logiquement l'un des chemins et briser la boucle physique.

2.3 MISE EN PLACE

2.3.1 SUR VMWARE

- Machine 1 : lan segment 1 (IP : 192.168.0.1)
- Machine 2 : lan segment 2 (IP : 192.168.0.2)
- Machine 3 : lan segment 3 (IP : 192.168.0.3)
- Switch 1 : lan segment 1, lan segment 4, lan segment 6
- Switch-2 : lan segment 2, lan segment 4, lan segment 5
- Switch 3 : lan segment 3, lan segment 5, lan segment 6

2.3.2 SUR LES MACHINES

On peut forcer l'activation du protocole avec :
(sur chaque Switch)

```
sudo mstpctl setforcevers bridge stp
```

- `mstpctl` est la commande qui sert à configurer mstpd (Multiple Spanning Tree daemon)
- `setforcevers` sert à spécifier une version du protocole, ici stp (defaut mstp)¹

NB: Le protocole STP est activé par défaut sur Cumulus, cette étape peut être considérée comme superflue

2.3.2.1 RÔLE ET ÉTAT DES PORTS

On peut voir l'état du protocole avec la commande `net show bridge spanning-tree` :

¹[mstpctl.8 | sourcecode - mstpctl](#)

```
cumulus@cumulus:mgmt:~$ net show bridge spanning-tree
Bridge info
  enabled      yes
  bridge id    8.000.00:0C:29:58:C1:04
  Priority:    32768
  Address:     00:0C:29:58:C1:04
  designated root 8.000.00:0C:29:16:76:00
  Priority:    32768
  Address:     00:0C:29:16:76:00

  root port    swp2 (#2)
  path cost    20000      internal path cost 0
  max age      20         bridge max age 20
  forward delay 15        bridge forward delay 15
  tx hold count 6         max hops 20
  hello time   2          ageing time 300
  force protocol version stp

INTERFACE  STATE  ROLE  EDGE
-----
swp1       forw  Desg
swp2       forw  Root
swp3       forw  Desg
cumulus@cumulus:mgmt:~$
```

Figure 2.3: Switch 1

```
cumulus@cumulus:mgmt:~$ net show bridge spanning-tree
Bridge info
  enabled      yes
  bridge id    8.000.00:0C:29:16:76:00
  Priority:    32768
  Address:     00:0C:29:16:76:00
  This bridge is root.

  designated root 8.000.00:0C:29:16:76:00
  Priority:    32768
  Address:     00:0C:29:16:76:00

  root port    none
  path cost    0          internal path cost 0
  max age      20         bridge max age 20
  forward delay 15        bridge forward delay 15
  tx hold count 6         max hops 20
  hello time   2          ageing time 300
  force protocol version stp

INTERFACE  STATE  ROLE  EDGE
-----
swp1       forw  Desg
swp2       forw  Desg
swp3       forw  Desg
cumulus@cumulus:mgmt:~$
```

Figure 2.4: Switch 2

```
cumulus@cumulus:mgmt:~$ net show bridge spanning-tree
Bridge info
  enabled      yes
  bridge id    8.000.00:0C:29:BD:54:5F
  Priority:    32768
  Address:     00:0C:29:BD:54:5F
  designated root 8.000.00:0C:29:16:76:00
  Priority:    32768
  Address:     00:0C:29:16:76:00

  root port    swp2 (#2)
  path cost    20000      internal path cost 0
  max age      20         bridge max age 20
  forward delay 15        bridge forward delay 15
  tx hold count 6         max hops 20
  hello time   2          ageing time 300
  force protocol version stp

INTERFACE  STATE  ROLE  EDGE
-----
swp1       forw  Desg
swp2       forw  Root
swp3       disc  Altn
cumulus@cumulus:mgmt:~$
```

Figure 2.5: Switch 3

- **Switch 1** : Son port swp2 est le Root Port (Root) (état Forwarding) avec un coût de 20000. Son port swp1 est Designated (Desg) (état Forwarding).
Le port swp3 du Switch 1 a été placé en rôle Alternate (Altn) et en état Discarding (disc). Il bloque le trafic pour casser la boucle, tout en restant à l'écoute d'éventuels changements de topologie
- **Switch-2 (Root Bridge)** : Ce switch a été élu pont racine (This bridge is root) car il possède l'adresse MAC la plus faible parmi ceux ayant la priorité par défaut (32768). Ses trois ports (swp1, swp2, swp3) sont en rôle Designated (Desg) et en état Forwarding (forw).
- **Switch 3** : Son port swp2 a été élu Root Port (Root) (état Forwarding) car c'est le chemin le plus court vers le Root Bridge. Les autres ports (swp1, swp3) sont en rôle Designated (Desg) (état Forwarding).

2.4 TEST STP

Pour simuler une panne, on va desactiver un des lien actif (swp2/Switch 1). Le lien passif (swp3/Switch-2) devrait alors prendre le relais pour assurer la communication.

On coupe le lien avec la commande `sudo ip link set swp2 down` :

Un ping est lancé sur une des machines.

```

root@debian:~# ping 192.168.0.2
PING 192.168.0.2 (192.168.0.2) 56(84) bytes of data.
64 bytes from 192.168.0.2: icmp_seq=1 ttl=64 time=5.83 ms
64 bytes from 192.168.0.2: icmp_seq=2 ttl=64 time=2.93 ms
64 bytes from 192.168.0.2: icmp_seq=3 ttl=64 time=2.90 ms
64 bytes from 192.168.0.2: icmp_seq=4 ttl=64 time=3.47 ms
64 bytes from 192.168.0.2: icmp_seq=5 ttl=64 time=3.14 ms
64 bytes from 192.168.0.2: icmp_seq=6 ttl=64 time=2.96 ms
64 bytes from 192.168.0.2: icmp_seq=7 ttl=64 time=2.92 ms
64 bytes from 192.168.0.2: icmp_seq=8 ttl=64 time=4.57 ms
64 bytes from 192.168.0.2: icmp_seq=9 ttl=64 time=3.29 ms
64 bytes from 192.168.0.2: icmp_seq=10 ttl=64 time=2.83 ms
64 bytes from 192.168.0.2: icmp_seq=40 ttl=64 time=6.46 ms
64 bytes from 192.168.0.2: icmp_seq=41 ttl=64 time=3.92 ms
64 bytes from 192.168.0.2: icmp_seq=42 ttl=64 time=3.57 ms
64 bytes from 192.168.0.2: icmp_seq=43 ttl=64 time=3.57 ms
64 bytes from 192.168.0.2: icmp_seq=44 ttl=64 time=3.79 ms
64 bytes from 192.168.0.2: icmp_seq=45 ttl=64 time=3.90 ms
64 bytes from 192.168.0.2: icmp_seq=46 ttl=64 time=4.83 ms
64 bytes from 192.168.0.2: icmp_seq=47 ttl=64 time=4.11 ms
64 bytes from 192.168.0.2: icmp_seq=48 ttl=64 time=3.76 ms
64 bytes from 192.168.0.2: icmp_seq=49 ttl=64 time=3.96 ms
64 bytes from 192.168.0.2: icmp_seq=50 ttl=64 time=3.62 ms
64 bytes from 192.168.0.2: icmp_seq=51 ttl=64 time=4.01 ms
^C
--- 192.168.0.2 ping statistics ---
51 packets transmitted, 22 received, 56.8627% packet loss, time 50748ms
rtt min/avg/max/mdev = 2.830/3.833/6.457/0.902 ms
root@debian:~# _

```

Figure 2.6: Ping Machine-1 ↔ Machine-2

```

debian@debian:~$ sudo tcpdump -i ens33 icmp
tcpdump: verbose output suppressed, use -v[v]... for full protocol decode
listening on ens33, link-type EN10MB (Ethernet), snapshot length 262144 bytes
18:04:43.393747 IP 192.168.0.1 > 192.168.0.2: ICMP echo request, id 2, seq 1, length 64
18:04:43.397559 IP 192.168.0.2 > 192.168.0.1: ICMP echo reply, id 2, seq 1, length 64
18:04:44.396294 IP 192.168.0.1 > 192.168.0.2: ICMP echo request, id 2, seq 2, length 64
18:04:44.399369 IP 192.168.0.2 > 192.168.0.1: ICMP echo reply, id 2, seq 2, length 64
18:04:45.398029 IP 192.168.0.1 > 192.168.0.2: ICMP echo request, id 2, seq 3, length 64
18:04:45.401557 IP 192.168.0.2 > 192.168.0.1: ICMP echo reply, id 2, seq 3, length 64
18:04:46.400300 IP 192.168.0.1 > 192.168.0.2: ICMP echo request, id 2, seq 4, length 64
18:04:46.403448 IP 192.168.0.2 > 192.168.0.1: ICMP echo reply, id 2, seq 4, length 64
18:04:47.402849 IP 192.168.0.1 > 192.168.0.2: ICMP echo request, id 2, seq 5, length 64
18:04:47.406275 IP 192.168.0.2 > 192.168.0.1: ICMP echo reply, id 2, seq 5, length 64
18:04:48.405013 IP 192.168.0.1 > 192.168.0.2: ICMP echo request, id 2, seq 6, length 64
18:04:48.408114 IP 192.168.0.2 > 192.168.0.1: ICMP echo reply, id 2, seq 6, length 64
18:04:49.406752 IP 192.168.0.1 > 192.168.0.2: ICMP echo request, id 2, seq 7, length 64
18:04:49.409651 IP 192.168.0.2 > 192.168.0.1: ICMP echo reply, id 2, seq 7, length 64
18:04:50.408961 IP 192.168.0.1 > 192.168.0.2: ICMP echo request, id 2, seq 8, length 64
18:04:50.411316 IP 192.168.0.2 > 192.168.0.1: ICMP echo reply, id 2, seq 8, length 64
18:04:51.411627 IP 192.168.0.1 > 192.168.0.2: ICMP echo request, id 2, seq 9, length 64
18:04:51.416874 IP 192.168.0.2 > 192.168.0.1: ICMP echo reply, id 2, seq 9, length 64
18:04:52.413392 IP 192.168.0.1 > 192.168.0.2: ICMP echo request, id 2, seq 10, length 64
18:04:52.417385 IP 192.168.0.2 > 192.168.0.1: ICMP echo reply, id 2, seq 10, length 64
18:04:53.416975 IP 192.168.0.1 > 192.168.0.2: ICMP echo request, id 2, seq 11, length 64
18:04:53.420266 IP 192.168.0.2 > 192.168.0.1: ICMP echo reply, id 2, seq 11, length 64
18:04:54.418193 IP 192.168.0.1 > 192.168.0.2: ICMP echo request, id 2, seq 12, length 64
18:04:54.422321 IP 192.168.0.2 > 192.168.0.1: ICMP echo reply, id 2, seq 12, length 64
18:04:55.420810 IP 192.168.0.1 > 192.168.0.2: ICMP echo request, id 2, seq 13, length 64
18:04:55.424572 IP 192.168.0.2 > 192.168.0.1: ICMP echo reply, id 2, seq 13, length 64
18:04:56.423247 IP 192.168.0.1 > 192.168.0.2: ICMP echo request, id 2, seq 14, length 64
18:04:56.427011 IP 192.168.0.2 > 192.168.0.1: ICMP echo reply, id 2, seq 14, length 64
18:04:57.425890 IP 192.168.0.1 > 192.168.0.2: ICMP echo request, id 2, seq 15, length 64
18:04:57.429779 IP 192.168.0.2 > 192.168.0.1: ICMP echo reply, id 2, seq 15, length 64
18:04:58.428237 IP 192.168.0.1 > 192.168.0.2: ICMP echo request, id 2, seq 16, length 64
18:04:58.431871 IP 192.168.0.2 > 192.168.0.1: ICMP echo reply, id 2, seq 16, length 64
18:04:59.430314 IP 192.168.0.1 > 192.168.0.2: ICMP echo request, id 2, seq 17, length 64
17:57:39.052477 IP 192.168.0.1 > 192.168.0.2: ICMP echo request, id 1, seq 25, length 64
17:57:31.076497 IP 192.168.0.1 > 192.168.0.2: ICMP echo request, id 1, seq 26, length 64
17:57:32.100361 IP 192.168.0.1 > 192.168.0.2: ICMP echo request, id 1, seq 27, length 64
17:57:33.124613 IP 192.168.0.1 > 192.168.0.2: ICMP echo request, id 1, seq 28, length 64
17:57:34.149033 IP 192.168.0.1 > 192.168.0.2: ICMP echo request, id 1, seq 29, length 64
17:57:35.172846 IP 192.168.0.1 > 192.168.0.2: ICMP echo request, id 1, seq 30, length 64
17:57:36.197055 IP 192.168.0.1 > 192.168.0.2: ICMP echo request, id 1, seq 31, length 64
17:57:37.220247 IP 192.168.0.1 > 192.168.0.2: ICMP echo request, id 1, seq 32, length 64
17:57:38.244985 IP 192.168.0.1 > 192.168.0.2: ICMP echo request, id 1, seq 33, length 64
17:57:39.268376 IP 192.168.0.1 > 192.168.0.2: ICMP echo request, id 1, seq 34, length 64
17:57:40.292765 IP 192.168.0.1 > 192.168.0.2: ICMP echo request, id 1, seq 35, length 64
17:57:41.316247 IP 192.168.0.1 > 192.168.0.2: ICMP echo request, id 1, seq 36, length 64
17:57:42.340711 IP 192.168.0.1 > 192.168.0.2: ICMP echo request, id 1, seq 37, length 64
17:57:43.364555 IP 192.168.0.1 > 192.168.0.2: ICMP echo request, id 1, seq 38, length 64
17:57:44.388339 IP 192.168.0.1 > 192.168.0.2: ICMP echo request, id 1, seq 39, length 64
17:57:45.412244 IP 192.168.0.1 > 192.168.0.2: ICMP echo request, id 1, seq 40, length 64
17:57:45.418639 IP 192.168.0.2 > 192.168.0.1: ICMP echo reply, id 1, seq 40, length 64
17:57:46.414702 IP 192.168.0.1 > 192.168.0.2: ICMP echo request, id 1, seq 41, length 64
17:57:46.418583 IP 192.168.0.2 > 192.168.0.1: ICMP echo reply, id 1, seq 41, length 64
17:57:47.417496 IP 192.168.0.1 > 192.168.0.2: ICMP echo request, id 1, seq 42, length 64
17:57:47.421015 IP 192.168.0.2 > 192.168.0.1: ICMP echo reply, id 1, seq 42, length 64
17:57:48.420808 IP 192.168.0.1 > 192.168.0.2: ICMP echo request, id 1, seq 43, length 64
17:57:48.423665 IP 192.168.0.2 > 192.168.0.1: ICMP echo reply, id 1, seq 43, length 64
17:57:49.421915 IP 192.168.0.1 > 192.168.0.2: ICMP echo request, id 1, seq 44, length 64
17:57:49.425666 IP 192.168.0.2 > 192.168.0.1: ICMP echo reply, id 1, seq 44, length 64

```

Figure 2.7: Tcpdump sur Machine-2

Observations : Le flux ICMP a été interrompu à la séquence `icmp_seq=10`. Le trafic n'a repris qu'à la séquence `icmp_seq=40`. On observe une perte : 51 packets transmitted, 22 received, 56.8627% packet loss, time 50748ms

Le lien passif à bien prit le relais mais avec un délais qui a permit une perte de paquets.

Explication : Avec le protocole STP classique (802.1D), lorsqu'un lien tombe, le réseau doit recalculer la topologie. Le port bloqué (swp3) doit passer par plusieurs états intermédiaires temporisés (Listening et Learning) avant de passer en Forwarding. Les timers par défaut (max age 20, forward delay 15) imposent un délai de convergence, ce qui explique les 30 requêtes ICMP perdues (de la séquence 11 à 39) avant la reprise de la connectivité.

En théorie, le délai est

$$2 * \text{forward delay} = 2 * 15s = 30s \quad (2.1)$$

(15s en *Listening*, puis 15s en *Learning*), ce qui coïncide avec les 30 paquets ICMP perdu, qui sont envoyés à une fréquence de (1 paquet) s^{-1} .

2.5 CONFIGURATION DE RSTP

Pour répondre à la contrainte de "reprise du trafic la plus rapide possible", nous modifions la version du protocole sur les switch pour utiliser RSTP (802.1w).

Pour cela, on force à nouveau un protocole spécifique : `sudo mstpcctl setforcevers bridge rstp`.

2.6 TEST RSTP

On coupe à nouveau le lien avec `sudo ip link set swp2 down`

```
64 bytes from 192.168.0.2: icmp_seq=23 ttl=64 time=3.55 ms
64 bytes from 192.168.0.2: icmp_seq=24 ttl=64 time=3.52 ms
64 bytes from 192.168.0.2: icmp_seq=25 ttl=64 time=3.61 ms
64 bytes from 192.168.0.2: icmp_seq=26 ttl=64 time=3.81 ms
64 bytes from 192.168.0.2: icmp_seq=27 ttl=64 time=3.83 ms
64 bytes from 192.168.0.2: icmp_seq=28 ttl=64 time=3.48 ms
64 bytes from 192.168.0.2: icmp_seq=29 ttl=64 time=3.93 ms
64 bytes from 192.168.0.2: icmp_seq=30 ttl=64 time=3.66 ms
64 bytes from 192.168.0.2: icmp_seq=31 ttl=64 time=3.59 ms
64 bytes from 192.168.0.2: icmp_seq=32 ttl=64 time=4.20 ms
64 bytes from 192.168.0.2: icmp_seq=33 ttl=64 time=3.75 ms
64 bytes from 192.168.0.2: icmp_seq=34 ttl=64 time=3.76 ms
64 bytes from 192.168.0.2: icmp_seq=35 ttl=64 time=3.79 ms
64 bytes from 192.168.0.2: icmp_seq=36 ttl=64 time=4.52 ms
64 bytes from 192.168.0.2: icmp_seq=37 ttl=64 time=3.88 ms
64 bytes from 192.168.0.2: icmp_seq=38 ttl=64 time=3.64 ms
64 bytes from 192.168.0.2: icmp_seq=39 ttl=64 time=3.59 ms
64 bytes from 192.168.0.2: icmp_seq=40 ttl=64 time=3.93 ms
64 bytes from 192.168.0.2: icmp_seq=41 ttl=64 time=3.99 ms
64 bytes from 192.168.0.2: icmp_seq=42 ttl=64 time=3.47 ms
64 bytes from 192.168.0.2: icmp_seq=43 ttl=64 time=3.78 ms
64 bytes from 192.168.0.2: icmp_seq=44 ttl=64 time=4.18 ms
64 bytes from 192.168.0.2: icmp_seq=45 ttl=64 time=3.34 ms
64 bytes from 192.168.0.2: icmp_seq=46 ttl=64 time=3.37 ms
64 bytes from 192.168.0.2: icmp_seq=47 ttl=64 time=5.74 ms
64 bytes from 192.168.0.2: icmp_seq=48 ttl=64 time=4.42 ms
64 bytes from 192.168.0.2: icmp_seq=49 ttl=64 time=3.73 ms
64 bytes from 192.168.0.2: icmp_seq=50 ttl=64 time=3.67 ms
64 bytes from 192.168.0.2: icmp_seq=51 ttl=64 time=5.03 ms
64 bytes from 192.168.0.2: icmp_seq=52 ttl=64 time=3.47 ms
64 bytes from 192.168.0.2: icmp_seq=53 ttl=64 time=3.88 ms
64 bytes from 192.168.0.2: icmp_seq=54 ttl=64 time=4.25 ms
64 bytes from 192.168.0.2: icmp_seq=55 ttl=64 time=3.51 ms
64 bytes from 192.168.0.2: icmp_seq=56 ttl=64 time=3.51 ms
64 bytes from 192.168.0.2: icmp_seq=57 ttl=64 time=3.73 ms
64 bytes from 192.168.0.2: icmp_seq=58 ttl=64 time=3.92 ms
64 bytes from 192.168.0.2: icmp_seq=59 ttl=64 time=3.78 ms
64 bytes from 192.168.0.2: icmp_seq=60 ttl=64 time=4.19 ms
64 bytes from 192.168.0.2: icmp_seq=61 ttl=64 time=3.31 ms
64 bytes from 192.168.0.2: icmp_seq=62 ttl=64 time=4.02 ms
64 bytes from 192.168.0.2: icmp_seq=63 ttl=64 time=3.36 ms
64 bytes from 192.168.0.2: icmp_seq=64 ttl=64 time=4.04 ms
64 bytes from 192.168.0.2: icmp_seq=65 ttl=64 time=3.57 ms
64 bytes from 192.168.0.2: icmp_seq=66 ttl=64 time=4.09 ms
64 bytes from 192.168.0.2: icmp_seq=67 ttl=64 time=3.94 ms
^C
--- 192.168.0.2 ping statistics ---
67 packets transmitted, 67 received, 0% packet loss, time 66153ms
rtt min/avg/max/medev = 2.965/3.860/5.735/0.509 ms
root@debian:~#
```

```
root@debian:~# sudo tcpdump -i ens33 icmp
tcpdump: verbose output suppressed, use -v|-vv for full protocol decode
listening on ens33, Link-type: EN10MB (Ethernet), snapshot length 262144 bytes
18:04:43.393747 IP 192.168.0.1 > 192.168.0.2: ICMP echo request, id 2, seq 1, length 64
18:04:43.397559 IP 192.168.0.2 > 192.168.0.1: ICMP echo reply, id 2, seq 1, length 64
18:04:44.396294 IP 192.168.0.1 > 192.168.0.2: ICMP echo request, id 2, seq 2, length 64
18:04:44.399369 IP 192.168.0.2 > 192.168.0.1: ICMP echo reply, id 2, seq 2, length 64
18:04:45.398929 IP 192.168.0.1 > 192.168.0.2: ICMP echo request, id 2, seq 3, length 64
18:04:45.403557 IP 192.168.0.2 > 192.168.0.1: ICMP echo reply, id 2, seq 3, length 64
18:04:46.408388 IP 192.168.0.1 > 192.168.0.2: ICMP echo request, id 2, seq 4, length 64
18:04:46.408488 IP 192.168.0.2 > 192.168.0.1: ICMP echo reply, id 2, seq 4, length 64
18:04:47.408284 IP 192.168.0.1 > 192.168.0.2: ICMP echo request, id 2, seq 5, length 64
18:04:47.408275 IP 192.168.0.2 > 192.168.0.1: ICMP echo reply, id 2, seq 5, length 64
18:04:48.408012 IP 192.168.0.1 > 192.168.0.2: ICMP echo request, id 2, seq 6, length 64
18:04:48.408114 IP 192.168.0.2 > 192.168.0.1: ICMP echo reply, id 2, seq 6, length 64
18:04:49.408752 IP 192.168.0.1 > 192.168.0.2: ICMP echo request, id 2, seq 7, length 64
18:04:49.408953 IP 192.168.0.2 > 192.168.0.1: ICMP echo reply, id 2, seq 7, length 64
18:04:50.408961 IP 192.168.0.1 > 192.168.0.2: ICMP echo request, id 2, seq 8, length 64
18:04:50.411516 IP 192.168.0.2 > 192.168.0.1: ICMP echo reply, id 2, seq 8, length 64
18:04:51.411627 IP 192.168.0.1 > 192.168.0.2: ICMP echo request, id 2, seq 9, length 64
18:04:51.416878 IP 192.168.0.2 > 192.168.0.1: ICMP echo reply, id 2, seq 9, length 64
18:04:52.413392 IP 192.168.0.1 > 192.168.0.2: ICMP echo request, id 2, seq 10, length 64
18:04:52.417385 IP 192.168.0.2 > 192.168.0.1: ICMP echo reply, id 2, seq 10, length 64
18:04:53.416975 IP 192.168.0.1 > 192.168.0.2: ICMP echo request, id 2, seq 11, length 64
18:04:53.420266 IP 192.168.0.2 > 192.168.0.1: ICMP echo reply, id 2, seq 11, length 64
18:04:54.418193 IP 192.168.0.1 > 192.168.0.2: ICMP echo request, id 2, seq 12, length 64
18:04:54.422321 IP 192.168.0.2 > 192.168.0.1: ICMP echo reply, id 2, seq 12, length 64
18:04:55.424818 IP 192.168.0.1 > 192.168.0.2: ICMP echo request, id 2, seq 13, length 64
18:04:55.428572 IP 192.168.0.2 > 192.168.0.1: ICMP echo reply, id 2, seq 13, length 64
18:04:56.432247 IP 192.168.0.1 > 192.168.0.2: ICMP echo request, id 2, seq 14, length 64
18:04:56.427931 IP 192.168.0.2 > 192.168.0.1: ICMP echo reply, id 2, seq 14, length 64
18:04:57.425898 IP 192.168.0.1 > 192.168.0.2: ICMP echo request, id 2, seq 15, length 64
18:04:57.429778 IP 192.168.0.2 > 192.168.0.1: ICMP echo reply, id 2, seq 15, length 64
18:04:58.428237 IP 192.168.0.1 > 192.168.0.2: ICMP echo request, id 2, seq 16, length 64
18:04:58.431871 IP 192.168.0.2 > 192.168.0.1: ICMP echo reply, id 2, seq 16, length 64
18:04:59.430314 IP 192.168.0.1 > 192.168.0.2: ICMP echo request, id 2, seq 17, length 64
```

Observations : Contrairement au test précédent, l'interruption a été quasi imperceptible. Tcpcdump et ping montrent une continuité quasi parfaite du trafic : 67 packets transmitted, 67 received, 0% packet loss, time 66153ms

Explication : Le RSTP réduit drastiquement le temps de convergence (généralement < 1 sec) car il n'utilise pas de timers fixes pour changer l'état des ports.

Il intègre un mécanisme de négociation active (*Proposal/Agreement*) entre les switch voisins.

Ainsi, lorsque le lien principal (swp2) tombe sur le Switch 1, le port alternatif (swp3) passe immé-

atement à l'état Forwarding sans devoir traverser les phases d'écoute et d'apprentissage, réduisant ainsi toute perte de paquets significative lors de notre test.²

²Understand Rapid Spanning Tree Protocol (802.1w)

TOLÉRANCE DE PANNE, COUCHE “ACCESS”

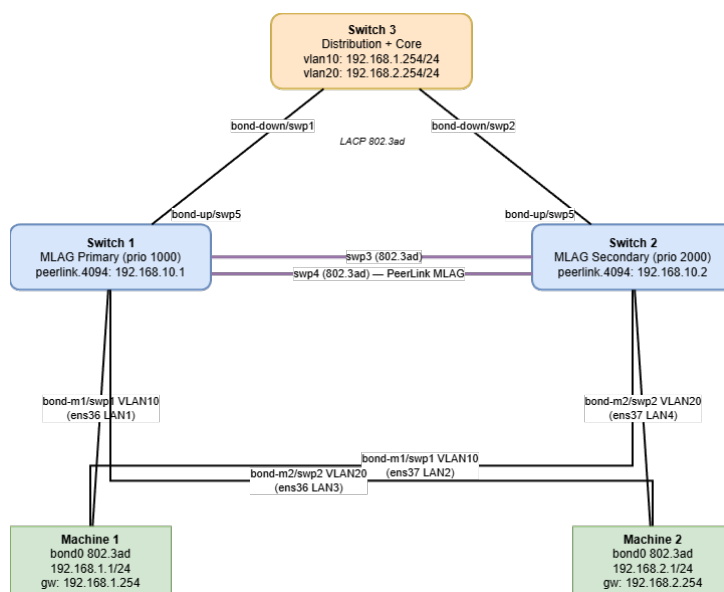


Figure 3.1: Maquette du réseau

3.1 RÉSUMÉ DES CONTRAINTES ET DES DEMANDES

- Assurer la tolérance aux pannes des commutateurs de la couche access (Switch-1 et Switch-2).
- En cas de défaillance de l'un des deux, le trafic entre Machine-1 et Machine-2 ne doit subir aucune interruption.
- Maximiser les performances : utilisation simultanée de tous les liens (actif/actif).
- Utilisation exclusive de Cumulus Linux ; aucun commutateur supplémentaire ; Machine-1 et Machine-2 ne peuvent pas être connectées directement à Switch-3.

Plan :

- Pour garantir la continuité de service en cas de panne d'un commutateur d'accès (Switch-1 ou Switch-2), chaque machine (Machine-1 et Machine-2) sera connectée physiquement et simultanément aux deux switches via une agrégation de liens utilisant le protocole LACP (mode 802.3ad).
- Pour maximiser les performances en permettant l'utilisation simultanée de tous les liens (mode actif-actif), une architecture MLAG (Multi-Chassis Link Aggregation) sera déployée entre le Switch-1 et le Switch-2. Cela nécessite la mise en place d'un "Peer Link" entre ces deux commutateurs pour synchroniser leurs états et agir comme un seul commutateur logique vis-à-vis des machines.

- Pour assurer la redondance vers la couche supérieure, une agrégation de liens sera également configurée entre notre nouveau cluster MLAG (Switch-1 et 2) et le routeur de distribution (Switch-3).
- Ajouter des liens $M1 \Leftrightarrow S2$ et $M1 \Leftrightarrow S1$
- Ajouter un *PeerLink* $Switch-1 \Leftrightarrow Switch-2$
- Agrégation de liens entre les Switch $\{1,2\} \Leftrightarrow Switch-3$

3.2 MISE EN PLACE

3.2.1 SUR VMWARE

- Machine-1 : connectée à LAN1 (vers SW1) et LAN2 (vers SW2)
- Machine-2 : connectée à LAN3 (vers SW1) et LAN4 (vers SW2)
- Switch-1 : connecté à LAN1, LAN3, LAN5, LAN6, LAN7
- Switch-2 : connecté à LAN2, LAN4, LAN5, LAN6, LAN8
- Switch-3 : connecté à LAN7, LAN8

3.2.2 SUR LES MACHINES

Sur les clients : Pour mettre en place l'agrégation de liens entre les clients et les switch, on utilise la configuration suivante sur les clients : Machine-1

```
...
auto bond0
iface bond0 inet static
    address 192.168.1.1/24
    gateway 192.168.1.254
    bond-slaves ens34 ens35
    bond-mode 802.3ad
    bond-miimon 100
    bond-lacp-rate 1
```

Machine-2

```
...
auto bond0
iface bond0 inet static
    address 192.168.2.1/24
    gateway 192.168.2.254
    bond-slaves ens36 ens37
    bond-mode 802.3ad
    bond-miimon 100
    bond-lacp-rate 1
```

Sur les switch :

Switch-1:

```
auto swp1
iface swp1
auto swp2
```

```
iface swp2
auto swp3
iface swp3
auto swp4
iface swp4
auto swp5
iface swp5

auto peerlink
iface peerlink
    bond-slaves swp3 swp4
    bond-mode 802.3ad

auto peerlink.4094
iface peerlink.4094
    address 192.168.10.1/24
    clagd-peer-ip 192.168.10.2
    clagd-sys-mac 44:38:39:FF:FF:FF
    clagd-priority 1000
    clagd-backup-ip 192.168.244.155

auto bond-m1
iface bond-m1
    bond-slaves swp1
    clag-id 1
    bridge-access 10

auto bond-m2
iface bond-m2
    bond-slaves swp2
    clag-id 2
    bridge-access 20

auto bond-up
iface bond-up
    bond-slaves swp5
    clag-id 3

auto bridge
iface bridge
    bridge-ports peerlink bond-m1 bond-m2 bond-up
    bridge-vlan-aware yes
    bridge-vids 10 20
```

Switch-2 : Seules les différences avec la configuration du Switch-1 sont montrées.

```
...
auto peerlink.4094
iface peerlink.4094
    address 192.168.10.2/24
    clagd-peer-ip 192.168.10.1
    clagd-sys-mac 44:38:39:FF:FF:FF
    clagd-priority 2000
```

```
clagd-backup-ip 192.168.244.154
...
```

Switch-3:

```
auto lo
iface lo inet loopback

auto eth0
iface eth0 inet dhcp
    vrf mgmt

auto mgmt
iface mgmt
    address 127.0.0.1/8
    vrf-table auto

auto bond-down
iface bond-down
    bond-slaves swp1 swp2
    bond-mode 802.3ad
    bond-miimon 100
    bond-lacp-rate 1

auto bridge
iface bridge
    bridge-ports bond-down
    bridge-vlan-aware yes
    bridge-vids 10 20

auto vlan10
iface vlan10
    vlan-id 10
    vlan-raw-device bridge
    address 192.168.1.254/24

auto vlan20
iface vlan20
    vlan-id 20
    vlan-raw-device bridge
    address 192.168.2.254/24
```

- `clagd-peer-ip` : Définit l'adresse IP du switch partenaire pour établir le canal de contrôle MLAG via le Peer Link.
- `clagd-sys-mac` : Adresse MAC système partagée par les deux commutateurs. C'est essentiel pour qu'ils apparaissent comme une seule et même entité logique (un seul switch) vis-à-vis des autres équipements (comme M1, M2 ou SW3).
- `clagd-priority` : Détermine le rôle du switch dans le cluster MLAG. Le switch avec la priorité la plus basse (ex: 1000 pour SW1) devient le "Primary", et l'autre (ex: 2000 pour SW2) devient le "Secondary".
- `clagd-backup-ip` : Définit une adresse IP de secours (généralement via le réseau de management Out-Of-Band). Si le Peer Link physique tombe, cette IP permet de vérifier si le switch partenaire

est totalement en panne ou si c’est juste le lien qui est coupé, évitant ainsi le problème du “split-brain” (où les deux switches penseraient être le maître).

- clag-id : Identifiant unique assigné à une interface agrégée (bond). Il doit être strictement identique sur les deux switches pour qu’ils synchronisent correctement le même lien LACP vers un équipement tiers.

On peut vérifier l’état du bond avec `net show clag`.

Switch-1:

```
cumulus@sw1:~$ net show clag
```

The peer is alive

Our Priority, ID, and Role: 1000 00:0c:29:45:30:fe primary

Peer Priority, ID, and Role: 2000 00:0c:29:64:76:96 secondary

Peer Interface and IP: peerlink.4094 192.168.10.2

Backup IP: 192.168.244.155 (active)

System MAC: 44:38:39:ff:ff:ff

CLAG Interfaces

Our Interface	Peer Interface	CLAG Id	Conflicts	Proto-Down
-----	-----	-----	-----	-----
bond-m1	bond-m1	1	-	-
bond-m2	bond-m2	2	-	-
bond-up	bond-up	3	-	-

The peer is alive : le Peer Link entre Switch-1 et Switch2 est fonctionnel.

Switch-1 est bien primaire (priorité 1000), Switch-2 secondaire (priorité 2000).

Les trois agrégats (bond-m1, bond-m2, bond-up) sont synchronisés sans conflit (colonne Conflicts vide).

Backup IP active : le canal de contrôle alternatif via le réseau de management est joignable.

Switch-2:

```
cumulus@sw2:~$ net show clag
```

The peer is alive

Our Priority, ID, and Role: 2000 00:0c:29:64:76:96 secondary

Peer Priority, ID, and Role: 1000 00:0c:29:45:30:fe primary

Peer Interface and IP: peerlink.4094 192.168.10.1

Backup IP: 192.168.244.154 (active)

System MAC: 44:38:39:ff:ff:ff

CLAG Interfaces

Our Interface	Peer Interface	CLAG Id	Conflicts	Proto-Down
-----	-----	-----	-----	-----
bond-m1	bond-m1	1	-	-
bond-m2	bond-m2	2	-	-
bond-up	bond-up	3	-	-

On peut vérifier l’état du bond sur les clients avec `cat /proc/net/bondinf/bond0.w`

```
Bonding Mode: IEEE 802.3ad Dynamic link aggregation
Transmit Hash Policy: layer2 (0)
MII Status: up
MII Polling Interval (ms): 100
```

```
802.3ad info
LACP active: on
LACP rate: fast
Aggregator selection policy (ad_select): stable
```

```
Slave Interface: ens36
MII Status: up | Speed: 1000 Mbps | Link Failure Count: 0
```

```
Slave Interface: ens37
MII Status: up | Speed: 1000 Mbps | Link Failure Count: 0
```

Identique sur la Machine-2.

On peut vérifier que les machines peuvent se ping :

```
Machine1:~# ping 192.168.2.1
64 bytes from 192.168.2.1: icmp_seq=1 ttl=63 time=2.87 ms
64 bytes from 192.168.2.1: icmp_seq=2 ttl=63 time=3.56 ms
64 bytes from 192.168.2.1: icmp_seq=3 ttl=63 time=2.51 ms
Le TTL de 63 confirme que le paquet est bien passé par un routeur.
```

3.3 TEST

En faisant une capture de trame avant de couper un switch, on peut observer les communications entre les switches:

```
▼ Actor State: 0x3f, LACP Activity, LACP Timeout, Aggregation, Synchronization, Collecting, Distributing
.... .1 = LACP Activity: Active
.... .1. = LACP Timeout: Short Timeout
.... .1.. = Aggregation: Aggregatable
.... 1... = Synchronization: In Sync
...1 .... = Collecting: Enabled
..1. .... = Distributing: Enabled
.0... .... = Defaulted: No
0... .... = Expired: No
```

Figure 3.2: communication normale entre les switch avant coupure

On lance un ping Machine-1 ↔ Machine-2, puis on éteint le Switch-1.

```
--- 192.168.2.1 ping statistics ---
99 packets transmitted, 97 received, 2.0202% packet loss, time 98255ms
rtt min/avg/max/mdev = 2.315/3.127/5.782/0.614 ms
```

Seulement 2 paquets perdus sur 99 lors de la coupure de Switch-1. Les 2 pertes correspondent au délai minimal de détection MII (bond-miimon 100 ms) et de basculement des flux du bond0 des machines vers le seul lien restant (ens37 vers Switch-2).

Redondance de Routeurs

4.1 Architecture

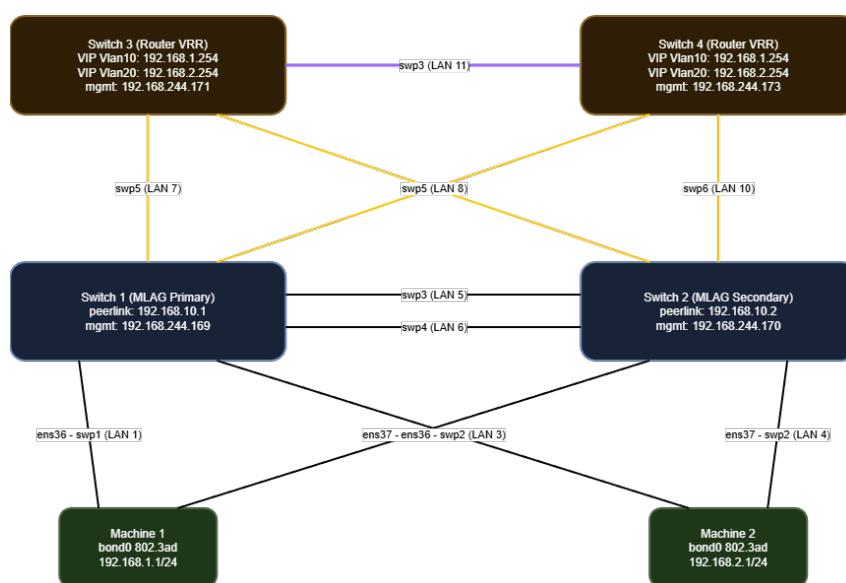


Figure 4.1: Maquette du réseau

Pour répondre au besoin de tolérance de panne sur la couche distribution (niveau 3) tout en maintenant la redondance de la couche accès (niveau 2), nous avons ajouté un quatrième switch (Switch-4) pour servir de secours au Switch-3.

Plan :

- Pour pallier la perte du routeur principal (Switch 3) au niveau de la couche distribution, un quatrième commutateur (Switch 4) sera ajouté à l'architecture pour servir de solution de secours (failover) tout en conservant la redondance de la couche accès.
- Pour assurer une reprise instantanée du trafic sans nécessiter de délai de bascule ni d'élection de maître, le protocole VRR (Virtual Router Redundancy), spécifique à Cumulus Linux, sera implémenté sur le Switch 3 et le Switch 4.
- Pour permettre un routage "Active-Active", la configuration VRR fera en sorte que les Switchs 3 et 4 partagent les mêmes adresses IP et adresses MAC virtuelles (192.168.1.254 et 192.168.2.254 avec les MAC 00:00:5E:00:01:10 / 00:00:5E:00:01:20). Ainsi, si le Switch 3 tombe, le Switch 4 routera les paquets instantanément car il répondra déjà aux mêmes requêtes ARP.
- Pour lier ce nouveau routeur de secours à l'infrastructure existante, de nouvelles agrégations de liens (Trunk de secours en mode 802.3ad) seront configurées pour relier les Switchs 1 et 2 vers le Switch 4.

4.2 Mise en Place

4.2.1 Sur VMware

- Machine-1 : LAN1 (vers SW1) et LAN2 (vers SW2)
- Machine-2 : LAN3 (vers SW1) et LAN4 (vers SW2)
- Switch 1 : LAN1, LAN3, LAN5, LAN6, LAN7
- Switch 2 : LAN2, LAN4, LAN5, LAN6, LAN8
- Switch 3 : LAN7, LAN8

4.2.2 Sur les Machines

Sur les Client

```
# Machine-1
auto ens36
iface ens36 inet manual
auto ens37
iface ens37 inet manual

auto bond0
iface bond0 inet static
address 192.168.1.1/24
gateway 192.168.1.254
bond-slaves ens36 ens37
bond-mode 802.3ad
bond-miimon 100
bond-lacp-rate 1
```

```
#Machine-2
auto ens36
iface ens36 inet manual
auto ens37
iface ens37 inet manual

auto bond0
iface bond0 inet static
address 192.168.2.1/24
gateway 192.168.2.254
bond-slaves ens36 ens37
bond-mode 802.3ad
bond-miimon 100
bond-lacp-rate 1
```

Sur les Switch :

```
#Switch 1
auto swp1
iface swp1
auto swp2
iface swp2
auto swp3
iface swp3
auto swp4
iface swp4
auto swp5
iface swp5

# Nouveau port vers le Switch 4
auto swp6
iface swp6

auto peerlink
iface peerlink
bond-slaves swp3 swp4
bond-mode 802.3ad

auto peerlink.4094
iface peerlink.4094
address 192.168.10.1/24
```

```
clagd-peer-ip 192.168.10.2
clagd-sys-mac 44:38:39:FF:FF:FF
clagd-priority 1000
clagd-backup-ip 192.168.244.170

auto bond-m1
iface bond-m1
bond-slaves swp1
clag-id 1
bridge-access 10

auto bond-m2
iface bond-m2
bond-slaves swp2
clag-id 2
bridge-access 20

# Agrégat vers Switch 3
auto bond-up
iface bond-up
bond-slaves swp5
clag-id 3

# Agrégat vers Switch 4 (Trunk de secours)
auto bond-up-sw4
iface bond-up-sw4
bond-slaves swp6
clag-id 4

auto bridge
iface bridge
bridge-ports peerlink bond-m1 bond-m2 bond-up bond-up-sw4
bridge-vlan-aware yes
bridge-vids 10 20
```

```
#Switch 2
auto swp1
iface swp1
auto swp2
iface swp2
auto swp3
iface swp3
auto swp4
iface swp4
auto swp5
iface swp5
# Nouveau port vers le Switch 4
auto swp6
iface swp6

auto peerlink
iface peerlink
bond-slaves swp3 swp4
bond-mode 802.3ad
```

```
auto peerlink.4094
iface peerlink.4094
address 192.168.10.2/24
clagd-peer-ip 192.168.10.1
clagd-sys-mac 44:38:39:FF:FF:FF
clagd-priority 2000
clagd-backup-ip 192.168.244.169

auto bond-m1
iface bond-m1
bond-slaves swp1
clag-id 1
bridge-access 10

auto bond-m2
iface bond-m2
bond-slaves swp2
clag-id 2
bridge-access 20

# Agrégat vers Switch 3
auto bond-up
iface bond-up
bond-slaves swp5
clag-id 3

# Agrégat vers Switch 4 (Trunk de secours)
auto bond-up-sw4
iface bond-up-sw4
bond-slaves swp6
clag-id 4

auto bridge
iface bridge
bridge-ports peerlink bond-m1 bond-m2 bond-up bond-up-sw4
bridge-vlan-aware yes
bridge-vids 10 20
```

```
#Switch 3
auto bond-down
iface bond-down
bond-slaves swp1 swp2
bond-mode 802.3ad
bond-miimon 100
bond-lacp-rate 1

# Lien Trunk vers le Switch 4
auto swp3
iface swp3

auto bridge
iface bridge
bridge-ports bond-down swp3
```

```

bridge-vlan-aware yes
bridge-vids 10 20

auto vlan10
iface vlan10
vlan-id 10
vlan-raw-device bridge
address 192.168.1.252/24
address-virtual 00:00:5E:00:01:10 192.168.1.254/24

auto vlan20
iface vlan20
vlan-id 20
vlan-raw-device bridge
address 192.168.2.252/24
address-virtual 00:00:5E:00:01:20 192.168.2.254/24

```

```

#Switch 4
auto bond-down
iface bond-down
bond-slaves swp1 swp2
bond-mode 802.3ad
bond-miimon 100
bond-lacp-rate 1

# Lien Trunk vers le Switch 3
auto swp3
iface swp3

auto bridge
iface bridge
bridge-ports bond-down swp3
bridge-vlan-aware yes
bridge-vids 10 20

auto vlan10
iface vlan10
vlan-id 10
vlan-raw-device bridge
address 192.168.1.253/24
address-virtual 00:00:5E:00:01:10 192.168.1.254/24

auto vlan20
iface vlan20
vlan-id 20
vlan-raw-device bridge
address 192.168.2.253/24
address-virtual 00:00:5E:00:01:20 192.168.2.254/24

```

- **address-virtual** : Cette directive assigne l'adresse MAC virtuelle (ex: 00:00:5E:00:01:10) et l'IP de la passerelle partagée (ex: 192.168.1.254/24) à l'interface VLAN. C'est elle qui permet le fonctionnement Actif-Actif sans nécessiter de protocole complexe comme VRRP.

4.3 Tests

4.3.1 Coupure dans la couche Access

Pour tester le réseau, on va lancer un ping M1 $\Leftrightarrow$ M2 puis éteindre le Switch-1.

```
64 bytes from 192.168.2.1: icmp_seq=45 ttl=63 time=5.57 ms
64 bytes from 192.168.2.1: icmp_seq=46 ttl=63 time=3.70 ms
64 bytes from 192.168.2.1: icmp_seq=50 ttl=63 time=4.43 ms
64 bytes from 192.168.2.1: icmp_seq=51 ttl=63 time=4.20 ms
64 bytes from 192.168.2.1: icmp_seq=52 ttl=63 time=4.60 ms
64 bytes from 192.168.2.1: icmp_seq=53 ttl=63 time=10.1 ms
64 bytes from 192.168.2.1: icmp_seq=54 ttl=63 time=3.01 ms
64 bytes from 192.168.2.1: icmp_seq=55 ttl=63 time=3.58 ms
64 bytes from 192.168.2.1: icmp_seq=56 ttl=63 time=5.96 ms
64 bytes from 192.168.2.1: icmp_seq=57 ttl=63 time=4.50 ms
64 bytes from 192.168.2.1: icmp_seq=58 ttl=63 time=3.63 ms
64 bytes from 192.168.2.1: icmp_seq=59 ttl=63 time=3.49 ms
64 bytes from 192.168.2.1: icmp_seq=60 ttl=63 time=4.18 ms
64 bytes from 192.168.2.1: icmp_seq=61 ttl=63 time=4.52 ms
64 bytes from 192.168.2.1: icmp_seq=62 ttl=63 time=3.47 ms
64 bytes from 192.168.2.1: icmp_seq=63 ttl=63 time=3.65 ms
64 bytes from 192.168.2.1: icmp_seq=64 ttl=63 time=3.84 ms
64 bytes from 192.168.2.1: icmp_seq=65 ttl=63 time=3.98 ms
^C
--- 192.168.2.1 ping statistics ---
65 packets transmitted, 62 received, 4.61538% packet loss, time 64201ms
rtt min/avg/max/mdev = 3.006/4.217/10.116/1.052 ms
debian@debian:~$ |
```

Figure 4.2: Ping de M1 à M2

Comme on peut le voir, la coupure du switch 1 entraîne un délai (~ 7ms) à la requête N°53.

```
16:27:24.545013 IP 192.168.1.1 > 192.168.2.1: ICMP echo request, id 3, seq 28, length 64
16:27:24.546776 IP 192.168.2.1 > 192.168.1.1: ICMP echo reply, id 3, seq 28, length 64
16:27:25.546791 IP 192.168.1.1 > 192.168.2.1: ICMP echo request, id 3, seq 29, length 64
16:27:25.546916 IP 192.168.2.1 > 192.168.1.1: ICMP echo request, id 3, seq 29, length 64
16:27:25.549949 IP 192.168.2.1 > 192.168.1.1: ICMP echo reply, id 3, seq 29, length 64
16:27:25.550035 IP 192.168.2.1 > 192.168.1.1: ICMP echo reply, id 3, seq 29, length 64
16:27:26.548925 IP 192.168.1.1 > 192.168.2.1: ICMP echo request, id 3, seq 30, length 64
16:27:26.548977 IP 192.168.1.1 > 192.168.2.1: ICMP echo request, id 3, seq 30, length 64
16:27:26.550619 IP 192.168.2.1 > 192.168.1.1: ICMP echo reply, id 3, seq 30, length 64
16:27:26.550649 IP 192.168.2.1 > 192.168.1.1: ICMP echo reply, id 3, seq 30, length 64
16:27:26.725871 ARP, Request who-has 192.168.1.254 tell 192.168.1.1, length 46
16:27:26.726015 ARP, Reply 192.168.1.254 is-at 00:00:5e:00:01:10, length 28
16:27:27.530721 ARP, Request who-has 192.168.1.1 tell 192.168.1.253, length 28
16:27:27.533665 ARP, Reply 192.168.1.1 is-at 00:0c:29:b3:53:cd, length 46
16:27:27.551981 IP 192.168.1.1 > 192.168.2.1: ICMP echo request, id 3, seq 31, length 64
16:27:27.552033 IP 192.168.1.1 > 192.168.2.1: ICMP echo request, id 3, seq 31, length 64
16:27:27.554032 IP 192.168.2.1 > 192.168.1.1: ICMP echo reply, id 3, seq 31, length 64
16:27:27.554115 IP 192.168.2.1 > 192.168.1.1: ICMP echo reply, id 3, seq 31, length 64
16:27:27.561002 ARP, Request who-has 192.168.2.254 tell 192.168.2.254, length 28
16:27:28.554504 IP 192.168.1.1 > 192.168.2.1: ICMP echo request, id 3, seq 32, length 64
16:27:28.554558 IP 192.168.1.1 > 192.168.2.1: ICMP echo request, id 3, seq 32, length 64
16:27:28.558392 IP 192.168.2.1 > 192.168.1.1: ICMP echo reply, id 3, seq 32, length 64
16:27:28.558562 IP 192.168.2.1 > 192.168.1.1: ICMP echo reply, id 3, seq 32, length 64
16:27:29.556814 IP 192.168.1.1 > 192.168.2.1: ICMP echo request, id 3, seq 33, length 64
16:27:29.556865 IP 192.168.1.1 > 192.168.2.1: ICMP echo request, id 3, seq 33, length 64
16:27:29.558777 IP 192.168.2.1 > 192.168.1.1: ICMP echo reply, id 3, seq 33, length 64
16:27:29.558848 IP 192.168.2.1 > 192.168.1.1: ICMP echo reply, id 3, seq 33, length 64
16:27:30.558728 IP 192.168.1.1 > 192.168.2.1: ICMP echo request, id 3, seq 34, length 64
16:27:30.558843 IP 192.168.1.1 > 192.168.2.1: ICMP echo request, id 3, seq 34, length 64
16:27:30.561677 IP 192.168.2.1 > 192.168.1.1: ICMP echo reply, id 3, seq 34, length 64
16:27:30.561760 IP 192.168.2.1 > 192.168.1.1: ICMP echo reply, id 3, seq 34, length 64
16:27:31.559742 IP 192.168.1.1 > 192.168.2.1: ICMP echo request, id 3, seq 35, length 64
16:27:31.559800 IP 192.168.1.1 > 192.168.2.1: ICMP echo request, id 3, seq 35, length 64
16:27:31.561982 IP 192.168.2.1 > 192.168.1.1: ICMP echo reply, id 3, seq 35, length 64
16:27:31.562066 IP 192.168.2.1 > 192.168.1.1: ICMP echo reply, id 3, seq 35, length 64
^C
67 packets captured
67 packets received by filter
0 packets dropped by kernel
cumulus@cumulus:mgmt~$ |
```

Figure 4.3: tcpdump sur un des clients

Le tcpdump confirme que les requêtes ICMP continuent d'être transférées et routées correctement à travers l'infrastructure restante.

4.3.2 Coupure dans la couche Distribution

En coupant le Switch-3, le Switch-4 prend immédiatement le relais. Grâce au protocole VRR, Switch-4 partage la même passerelle (IP et adresse MAC virtuelle) que Switch-3. Par conséquent, les caches ARP des hôtes n'ont pas besoin d'expirer ou de se mettre à jour ; le trafic est instantanément traité par l'interface bond-down de Switch-4.

```
debian@debian:~$ ping 192.168.2.1
PING 192.168.2.1 (192.168.2.1) 56(84) bytes of data.
64 bytes from 192.168.2.1: icmp_seq=1 ttl=63 time=4.03 ms
64 bytes from 192.168.2.1: icmp_seq=2 ttl=63 time=5.50 ms
64 bytes from 192.168.2.1: icmp_seq=3 ttl=63 time=3.87 ms
64 bytes from 192.168.2.1: icmp_seq=4 ttl=63 time=3.93 ms
64 bytes from 192.168.2.1: icmp_seq=5 ttl=63 time=5.26 ms
64 bytes from 192.168.2.1: icmp_seq=6 ttl=63 time=5.80 ms
64 bytes from 192.168.2.1: icmp_seq=7 ttl=63 time=4.41 ms
64 bytes from 192.168.2.1: icmp_seq=8 ttl=63 time=3.49 ms
64 bytes from 192.168.2.1: icmp_seq=9 ttl=63 time=3.92 ms
64 bytes from 192.168.2.1: icmp_seq=10 ttl=63 time=4.10 ms
64 bytes from 192.168.2.1: icmp_seq=11 ttl=63 time=6.99 ms
64 bytes from 192.168.2.1: icmp_seq=12 ttl=63 time=6.21 ms
64 bytes from 192.168.2.1: icmp_seq=13 ttl=63 time=5.26 ms
64 bytes from 192.168.2.1: icmp_seq=14 ttl=63 time=4.16 ms
64 bytes from 192.168.2.1: icmp_seq=15 ttl=63 time=4.03 ms
64 bytes from 192.168.2.1: icmp_seq=16 ttl=63 time=4.05 ms
64 bytes from 192.168.2.1: icmp_seq=17 ttl=63 time=3.38 ms
64 bytes from 192.168.2.1: icmp_seq=22 ttl=63 time=15.7 ms
64 bytes from 192.168.2.1: icmp_seq=23 ttl=63 time=5.45 ms
64 bytes from 192.168.2.1: icmp_seq=24 ttl=63 time=4.51 ms
64 bytes from 192.168.2.1: icmp_seq=25 ttl=63 time=4.46 ms
64 bytes from 192.168.2.1: icmp_seq=26 ttl=63 time=7.07 ms
64 bytes from 192.168.2.1: icmp_seq=27 ttl=63 time=4.63 ms
64 bytes from 192.168.2.1: icmp_seq=28 ttl=63 time=3.83 ms
64 bytes from 192.168.2.1: icmp_seq=29 ttl=63 time=5.38 ms
64 bytes from 192.168.2.1: icmp_seq=30 ttl=63 time=4.25 ms
64 bytes from 192.168.2.1: icmp_seq=31 ttl=63 time=4.57 ms
64 bytes from 192.168.2.1: icmp_seq=32 ttl=63 time=6.31 ms
64 bytes from 192.168.2.1: icmp_seq=33 ttl=63 time=3.94 ms
64 bytes from 192.168.2.1: icmp_seq=34 ttl=63 time=6.17 ms
64 bytes from 192.168.2.1: icmp_seq=35 ttl=63 time=4.63 ms
^C
--- 192.168.2.1 ping statistics ---
35 packets transmitted, 31 received, 11.4286% packet loss, time 34166ms
rtt min/avg/max/mdev = 3.382/5.136/15.659/2.158 ms
debian@debian:~$
```

Figure 4.4: Ping de M1 à M2

```

cumulus@cumulus-mgmt:~$ sudo tcpdump -i bridge arp or icmp -n
tcpdump: verbose output suppressed, use -v or -vv for full protocol decode
listening on bridge, link-type EN10MB (Ethernet), capture size 262144 bytes
16:26:57.395048 IP 192.168.1.1 > 192.168.2.1: ICMP echo request, id 3, seq 1, length 64
16:27:09.813904 ARP, Request who-has 192.168.1.1 tell 192.168.1.252, length 46
16:27:18.532322 IP 192.168.1.1 > 192.168.2.1: ICMP echo request, id 3, seq 22, length 64
16:27:18.532526 ARP, Request who-has 192.168.2.1 tell 192.168.2.253, length 28
16:27:18.534993 ARP, Reply 192.168.2.1 is-at 00:0c:29:bc:09:92, length 46
16:27:18.535065 IP 192.168.1.1 > 192.168.2.1: ICMP echo request, id 3, seq 22, length 64
16:27:18.538221 IP 192.168.2.1 > 192.168.1.1: ICMP echo reply, id 3, seq 22, length 64
16:27:18.538321 ARP, Request who-has 192.168.1.1 tell 192.168.1.253, length 28
16:27:18.543788 ARP, Reply 192.168.1.1 is-at 00:0c:29:b3:53:cd, length 46
16:27:18.543811 IP 192.168.2.1 > 192.168.1.1: ICMP echo reply, id 3, seq 22, length 64
16:27:19.534504 IP 192.168.1.1 > 192.168.2.1: ICMP echo request, id 3, seq 23, length 64
16:27:19.534724 IP 192.168.1.1 > 192.168.2.1: ICMP echo request, id 3, seq 23, length 64
16:27:19.537111 IP 192.168.2.1 > 192.168.1.1: ICMP echo reply, id 3, seq 23, length 64
16:27:19.537141 IP 192.168.2.1 > 192.168.1.1: ICMP echo reply, id 3, seq 23, length 64
16:27:20.535983 IP 192.168.1.1 > 192.168.2.1: ICMP echo request, id 3, seq 24, length 64
16:27:20.536896 IP 192.168.1.1 > 192.168.2.1: ICMP echo request, id 3, seq 24, length 64
16:27:20.537918 IP 192.168.2.1 > 192.168.1.1: ICMP echo reply, id 3, seq 24, length 64
16:27:20.538050 IP 192.168.2.1 > 192.168.1.1: ICMP echo reply, id 3, seq 24, length 64
16:27:21.538107 IP 192.168.1.1 > 192.168.2.1: ICMP echo request, id 3, seq 25, length 64
16:27:21.538240 IP 192.168.1.1 > 192.168.2.1: ICMP echo request, id 3, seq 25, length 64
16:27:21.540407 IP 192.168.2.1 > 192.168.1.1: ICMP echo reply, id 3, seq 25, length 64
16:27:21.540492 IP 192.168.2.1 > 192.168.1.1: ICMP echo reply, id 3, seq 25, length 64
16:27:22.540338 IP 192.168.1.1 > 192.168.2.1: ICMP echo request, id 3, seq 26, length 64
16:27:22.540397 IP 192.168.1.1 > 192.168.2.1: ICMP echo request, id 3, seq 26, length 64
16:27:22.544977 IP 192.168.2.1 > 192.168.1.1: ICMP echo reply, id 3, seq 26, length 64
16:27:22.545047 IP 192.168.2.1 > 192.168.1.1: ICMP echo reply, id 3, seq 26, length 64
16:27:23.542570 IP 192.168.1.1 > 192.168.2.1: ICMP echo request, id 3, seq 27, length 64
16:27:23.542647 IP 192.168.1.1 > 192.168.2.1: ICMP echo request, id 3, seq 27, length 64
16:27:23.544497 IP 192.168.2.1 > 192.168.1.1: ICMP echo reply, id 3, seq 27, length 64
16:27:23.544533 IP 192.168.2.1 > 192.168.1.1: ICMP echo reply, id 3, seq 27, length 64
16:27:24.544939 IP 192.168.1.1 > 192.168.2.1: ICMP echo request, id 3, seq 28, length 64
16:27:24.545015 IP 192.168.1.1 > 192.168.2.1: ICMP echo request, id 3, seq 28, length 64
16:27:24.546702 IP 192.168.2.1 > 192.168.1.1: ICMP echo reply, id 3, seq 28, length 64
16:27:24.546776 IP 192.168.2.1 > 192.168.1.1: ICMP echo reply, id 3, seq 28, length 64
16:27:25.546791 IP 192.168.1.1 > 192.168.2.1: ICMP echo request, id 3, seq 29, length 64
16:27:25.546916 IP 192.168.1.1 > 192.168.2.1: ICMP echo request, id 3, seq 29, length 64
18:21:57.898422 IP 192.168.2.1 > 192.168.1.1: ICMP echo reply, id 2, seq 17, length 64
18:21:58.901084 IP 192.168.2.1 > 192.168.1.1: ICMP echo reply, id 2, seq 18, length 64
18:21:59.904174 IP 192.168.2.1 > 192.168.1.1: ICMP echo reply, id 2, seq 19, length 64
18:22:00.905139 IP 192.168.2.1 > 192.168.1.1: ICMP echo reply, id 2, seq 20, length 64
18:22:01.907321 IP 192.168.2.1 > 192.168.1.1: ICMP echo reply, id 2, seq 21, length 64
18:22:02.910783 IP 192.168.2.1 > 192.168.1.1: ICMP echo reply, id 2, seq 22, length 64
18:22:03.913025 IP 192.168.2.1 > 192.168.1.1: ICMP echo reply, id 2, seq 23, length 64
18:22:04.913811 IP 192.168.2.1 > 192.168.1.1: ICMP echo reply, id 2, seq 24, length 64
18:22:05.914905 IP 192.168.2.1 > 192.168.1.1: ICMP echo reply, id 2, seq 25, length 64
18:22:06.917215 IP 192.168.2.1 > 192.168.1.1: ICMP echo reply, id 2, seq 26, length 64
18:22:07.920838 IP 192.168.2.1 > 192.168.1.1: ICMP echo reply, id 2, seq 27, length 64
18:22:08.922983 IP 192.168.2.1 > 192.168.1.1: ICMP echo reply, id 2, seq 28, length 64
18:22:09.925136 IP 192.168.2.1 > 192.168.1.1: ICMP echo reply, id 2, seq 29, length 64
18:22:10.927051 IP 192.168.2.1 > 192.168.1.1: ICMP echo reply, id 2, seq 30, length 64
18:22:11.929321 IP 192.168.2.1 > 192.168.1.1: ICMP echo reply, id 2, seq 31, length 64
18:22:12.931606 IP 192.168.2.1 > 192.168.1.1: ICMP echo reply, id 2, seq 32, length 64
18:22:13.932786 IP 192.168.2.1 > 192.168.1.1: ICMP echo reply, id 2, seq 33, length 64
18:22:14.935574 IP 192.168.2.1 > 192.168.1.1: ICMP echo reply, id 2, seq 34, length 64
18:22:15.937911 IP 192.168.2.1 > 192.168.1.1: ICMP echo reply, id 2, seq 35, length 64
18:22:16.939268 IP 192.168.2.1 > 192.168.1.1: ICMP echo reply, id 2, seq 36, length 64
18:22:17.942473 IP 192.168.2.1 > 192.168.1.1: ICMP echo reply, id 2, seq 37, length 64
18:22:18.943983 IP 192.168.2.1 > 192.168.1.1: ICMP echo reply, id 2, seq 38, length 64
18:22:19.946842 IP 192.168.2.1 > 192.168.1.1: ICMP echo reply, id 2, seq 39, length 64
18:22:20.950038 IP 192.168.2.1 > 192.168.1.1: ICMP echo reply, id 2, seq 40, length 64
18:22:21.955835 IP 192.168.2.1 > 192.168.1.1: ICMP echo reply, id 2, seq 41, length 64
18:22:22.958959 IP 192.168.2.1 > 192.168.1.1: ICMP echo reply, id 2, seq 42, length 64
18:22:23.959521 IP 192.168.2.1 > 192.168.1.1: ICMP echo reply, id 2, seq 43, length 64
18:22:24.961767 IP 192.168.2.1 > 192.168.1.1: ICMP echo reply, id 2, seq 44, length 64
18:22:25.964766 IP 192.168.2.1 > 192.168.1.1: ICMP echo reply, id 2, seq 45, length 64
18:22:26.967037 IP 192.168.2.1 > 192.168.1.1: ICMP echo reply, id 2, seq 46, length 64
18:22:27.969286 IP 192.168.2.1 > 192.168.1.1: ICMP echo reply, id 2, seq 47, length 64
18:22:28.982675 IP 192.168.2.1 > 192.168.1.1: ICMP echo reply, id 2, seq 48, length 64
18:22:30.006479 IP 192.168.2.1 > 192.168.1.1: ICMP echo reply, id 2, seq 49, length 64
^C
49 packets captured
49 packets received by filter
0 packets dropped by kernel
cubian@cubian:~$

```

Figure 4.5: tcpdump sur un des clients

L'analyse de la trace réseau montre bien les requêtes ICMP qui s'enchaînent, et on voit également les requêtes ARP Request *who-has 192.168.1.1* traitées par les équipements sans interruption fatale du flux. La redondance du routeur est donc fonctionnelle.

Le code pour générer ce document est disponible à [Github.com/PaulVerot03/ASRA-Cumulus](https://github.com/PaulVerot03/ASRA-Cumulus)